

# Pittsburgh Winter Salt Truck Route Optimization System

## Complete User Guide & Documentation

---

### TABLE OF CONTENTS

1. [System Overview](#)
  2. [Installation & Setup](#)
  3. [How to Run the System](#)
  4. [Understanding the Output](#)
  5. [Customizing for Your City](#)
  6. [Updating Data for Accuracy](#)
  7. [Troubleshooting](#)
  8. [Advanced Configuration](#)
  9. [Maintenance & Updates](#)
- 

## SYSTEM OVERVIEW

### What Does This System Do?

This system optimizes salt truck routes for winter storm response in Pittsburgh. It:

- ✓ **Downloads real-time data** from OpenStreetMap, National Weather Service, and traffic APIs
- ✓ **Prioritizes roads** based on importance (emergency routes, schools, hospitals)
- ✓ **Assigns trucks** to routes efficiently across 5 depot facilities
- ✓ **Calculates costs** (salt, fuel, labor) for budgeting
- ✓ **Generates professional reports** (Excel spreadsheets and interactive maps)

### Who Should Use This?

- **Operations Managers** - Plan fleet deployment before storms
- **Dispatchers** - Coordinate trucks during events
- **Budget Directors** - Estimate costs for winter operations
- **Public Affairs** - Communicate coverage to residents
- **City Planners** - Optimize depot locations and fleet size

## Key Features

| Feature                 | Description                        | Benefit                                          |
|-------------------------|------------------------------------|--------------------------------------------------|
| Real-time Weather       | Pulls current conditions from NWS  | Adjusts salt rates for temperature/precipitation |
| OSM Integration         | Downloads Pittsburgh road network  | Always up-to-date with new streets               |
| Jurisdiction Filtering  | Excludes PennDOT state highways    | Only shows city responsibility roads             |
| Multi-depot Support     | Coordinates 5 facilities           | Realistic truck assignments                      |
| Cost Analysis           | Calculates salt, fuel, labor costs | Budget planning and reporting                    |
| Multiple Visualizations | Excel + 9 different maps           | Serves different audiences                       |

## INSTALLATION & SETUP

### Prerequisites

You need:

- **Windows 10/11** (or Mac/Linux with minor path adjustments)
- **Anaconda** or **Python 3.11+** installed
- **Internet connection** (for downloading map data)
- **Excel** or Excel viewer (to open .xlsx reports)
- **Web browser** (Chrome, Firefox, Edge - for maps)

### Step-by-Step Installation

#### 1. Create Python Environment

Open **Anaconda Prompt** and run:

```
bash

conda create -n salttrucks python=3.11 -y
conda activate salttrucks
```

#### 2. Install Required Packages

```
bash
```

```
# Core packages
```

```
conda install -c conda-forge osmnx spyder-kernels -y
```

```
# Additional packages
```

```
pip install openpyxl folium requests scipy
```

### 3. Download the Scripts

Save these 3 files to your Desktop (or any folder):

- `PGH_Salt_Trucks_FINAL.py` - Main optimization script
- `generate_excel_report.py` - Excel report generator
- `generate_maps.py` - Map visualization generator

### 4. Verify Installation

Test that everything works:

```
bash
```

```
python -c "import osmnx, folium, openpyxl; print('✓ All packages installed!)"
```

You should see: `✓ All packages installed!`

---

## HOW TO RUN THE SYSTEM

### Basic Workflow

Step 1: Run Main Script → Generates JSON data file

Step 2: Run Excel Generator → Creates spreadsheet report

Step 3: Run Map Generator → Creates interactive maps

### Step 1: Generate Route Optimization

#### Open Spyder

```
bash
```

```
conda activate salttrucks
```

```
spyder
```

## Open Main Script

In Spyder: **File** → **Open** → Select **PGH\_Salt\_Trucks\_FINAL.py**

## Run the Script

Click the green **Run** button (or press **F5**)

## What Happens:

1. Downloads weather from National Weather Service (~5 seconds)
2. Downloads Pittsburgh roads from OpenStreetMap (~30 seconds)
3. Filters out state/county roads (only city responsibility)
4. Prioritizes roads (emergency routes, schools, hospitals)
5. Optimizes 68 truck routes across 5 depots (~2-3 minutes)
6. Saves results to Desktop

**Expected Runtime:** 3-5 minutes total

## Output Files Created:

- **optimization\_results.json** - Complete route data (used by other scripts)

## Step 2: Generate Excel Report

In Anaconda Prompt:

```
bash  
  
cd Desktop  
python generate_excel_report.py
```

## Output Created:

- **salt\_truck\_report.xlsx** - 6-sheet Excel workbook

## Open in Excel to view:

- Executive Summary
- All Truck Routes (sortable table)
- Priority Analysis
- Cost Breakdown
- Depot Performance

- Detailed Routes

### Step 3: Generate Maps

In Anaconda Prompt:

```
bash  
  
cd Desktop  
python generate_maps.py
```

#### Output Created:

- `map_full_fleet.html` - All 68 trucks (interactive)
- `map_priority_roads.html` - High-priority roads only
- `map_public_simple.html` - Clean version for residents
- `map_depot_[name].html` - 5 depot-specific maps
- `map_coverage_heatmap.html` - Coverage density

Open any `.html` file in your web browser to view

---

## UNDERSTANDING THE OUTPUT

### JSON File (`optimization_results.json`)

This is the raw data file. You don't need to open it directly - it's used by the Excel and map generators.

#### Contains:

- Complete route for each truck
- Every road segment with GPS coordinates
- Timing, distance, salt usage per segment
- Weather conditions
- Summary statistics

### Excel Report Structure

#### Sheet 1: Executive Summary

**Use Case:** Quick overview for leadership

**Key Metrics:**

- Fleet size and deployment time
- Weather conditions and severity
- Total costs and completion estimate
- Priority road distribution

**Best For:** Morning briefings, status reports

## **Sheet 2: All Truck Routes**

**Use Case:** Operations management

**Features:**

- Sortable by any column (truck, depot, distance, etc.)
- Filter to see specific depots or time ranges
- Compare truck workloads

**Best For:** Ensuring balanced workload, identifying issues

## **Sheet 3: Priority Analysis**

**Use Case:** Verify critical coverage

**Features:**

- Lists all high-priority roads
- Shows which truck covers each
- Service time and salt usage

**Best For:** Confirming hospitals, schools, fire stations are covered

## **Sheet 4: Cost Analysis**

**Use Case:** Budget planning and reporting

**Features:**

- Assumptions clearly stated (you can adjust)
- Breakdown: salt, fuel, labor
- Per-unit costs (per mile, per road, per truck)

**Best For:** Finance reports, budget justification

## **Sheet 5: Depot Performance**

**Use Case:** Compare facility efficiency

**Features:**

- Side-by-side depot comparison
- Identify underutilized or overburdened facilities

**Best For:** Long-term planning, fleet rebalancing

## **Sheet 6: Detailed Routes**

**Use Case:** Operational debugging

**Features:**

- Turn-by-turn breakdown for first 10 trucks
- See breaks, refills, individual roads

**Best For:** Training, verifying routes on the ground

## **Map Visualizations**

### **Full Fleet Map (map\_full\_fleet.html)**

**Audience:** Operations managers, dispatchers

**Features:**

- Toggle individual trucks on/off
- Different color per truck
- Click routes for details
- See all depot locations

**When to Use:** Coordinating fleet during event

### **Priority Roads Map (map\_priority\_roads.html)**

**Audience:** Operations managers, city officials

**Features:**

- Red = Emergency routes
- Orange = High priority
- Yellow = Medium priority
- Decluttered view

**When to Use:** Verify critical infrastructure coverage

## **Public Simple Map (map\_public\_simple.html)**

**Audience:** Residents, media, public

**Features:**

- Clean, easy-to-understand
- No technical details
- Shows coverage estimate
- Large info box with completion time

**When to Use:** Social media posts, website updates, press releases

## **Depot Maps (map\_depot\_[name].html)**

**Audience:** Individual facility managers

**Features:**

- Shows only trucks from one depot
- Focused view on their service area
- Easier for briefing individual teams

**When to Use:** Morning briefings at each facility

## **Heat Map (map\_coverage\_heatmap.html)**

**Audience:** Planners, analysts

**Features:**

- Shows coverage density
- Identifies gaps or overlaps
- No individual routes, just patterns

**When to Use:** Long-term planning, optimization analysis

---

## **CUSTOMIZING FOR YOUR CITY**

### **If You're NOT Pittsburgh**

This system can be adapted to any city! Here's how:

#### **1. Update City Information**

Open `PGH_Salt_Trucks_FINAL.py` in a text editor and find:



```
python
```

```
# Lines 20-21
```

```
PITTSBURGH_COORDS = (40.4406, -79.9959)
```

```
PITTSBURGH_BBOX = (40.361, 40.501, -80.095, -79.865)
```

### Change to your city:

```
python
```

```
CITY_COORDS = (YOUR_LATITUDE, YOUR_LONGITUDE)
```

```
CITY_BBOX = (SOUTH_LAT, NORTH_LAT, WEST_LON, EAST_LON)
```

### How to find coordinates:

1. Go to [Google Maps](#)
2. Right-click your city center → "What's here?"
3. Copy the latitude/longitude

### For bounding box:

- Draw a rectangle around your city
- Get coordinates of SW corner (South lat, West lon)
- Get coordinates of NE corner (North lat, East lon)

## 2. Update Depot Locations

Find the depot section (around line 1102):

```
python
```

```
depots = [
```

```
    Depot("YOUR_DEPOT_1", lat, lon, osm_node_id, trucks=20, refill_time=15),
```

```
    Depot("YOUR_DEPOT_2", lat, lon, osm_node_id, trucks=15, refill_time=15),
```

```
    # Add more as needed
```

```
]
```

### To find OSM node ID:

1. Go to [OpenStreetMap.org](#)
2. Search your depot address
3. Click "Show on map"

4. The URL will have: `...node/XXXXXXXXXX` - that's your node ID

### 3. Update Jurisdiction Filters

If your city has state/county roads to exclude:

Find lines 32-60 and update:

```
python

EXCLUDE_HIGHWAY_TYPES = [
    'motorway',    # Your state interstates
    'motorway_link',
    'trunk',       # Your state routes
]

STATE_MAINTAINED_ROADS = {
    'Interstate 90', # Replace with your roads
    'US Highway 50',
    'State Route 1',
    # etc.
}
```

#### How to identify state roads:

- Contact your state DOT
  - Check maintenance responsibility maps
  - Ask your public works director
- 

## UPDATING DATA FOR ACCURACY

### Weather Integration

**Current Source:** National Weather Service (NWS) API

**Location:** Line 182 in main script

```
python

def fetch_weather_data():
    # Uses Pittsburgh NWS grid point
    url = "https://api.weather.gov/gridpoints/PBZ/75,65/forecast"
```

#### To Update for Your City:

1. Go to [weather.gov](https://weather.gov)
2. Enter your address
3. Click on current forecast
4. In the URL, find: `/gridpoints/XXX/##,##/`
5. Replace `PBZ/75,65` with your grid point

**Alternative:** Use other weather APIs

- OpenWeatherMap
- WeatherAPI
- Dark Sky (Apple Weather)

Just replace the `fetch_weather_data()` function to call your preferred API.

## Road Priority Data

**Current Method:** Automated based on road type and proximity to critical infrastructure

**Location:** Lines 480-580 (POI identification)

**To Improve Accuracy:**

### Option 1: Add Known Priority Roads (Manual)

Find line ~500 and add:

```
python

MANUAL_PRIORITY_ROADS = {
    "Main Street": 1,    # Emergency
    "Hospital Drive": 1, # Emergency
    "School Street": 2,  # High priority
    # Add your city's critical roads
}
```

Then in the priority assignment (line ~425):

```
python
```

```

# Check manual overrides first
if road_name in MANUAL_PRIORITY_ROADS:
    priority = MANUAL_PRIORITY_ROADS[road_name]
else:
    # Use automated priority logic
    priority = self._calculate_automatic_priority(road)

```

## Option 2: Import from City Database

If your city has a road priority database (CSV, Excel, etc.):

```

python

import pandas as pd

# At start of script
priority_db = pd.read_csv("city_road_priorities.csv")
# CSV should have columns: road_name, priority

# In priority assignment
def get_priority_from_database(road_name):
    match = priority_db[priority_db['road_name'] == road_name]
    if not match.empty:
        return match['priority'].values[0]
    return 4 # Default low priority

```

## Fleet Size Optimization

**Current Setup:** 68 trucks across 5 depots

**Location:** Lines 1102-1110

**To Adjust:**

```

python

depots = [
    Depot("Depot A", lat, lon, node, trucks=15), # Change number here
    Depot("Depot B", lat, lon, node, trucks=20),
    # ...
]

```

## Guidelines:

- **Small city:** 1 truck per 8-12 centerline miles

- **Medium city:** 1 truck per 10-15 centerline miles
- **Large city:** 1 truck per 12-18 centerline miles

The script will show you how many centerline miles you have after downloading roads.

## Cost Assumptions

**Location:** Excel generator, line ~255

```
python

# Current assumptions
salt_cost_per_ton = 50
fuel_cost_per_gallon = 3.50
mpg = 6
driver_wage_per_hour = 35
```

## To Update with Your City's Costs:

Open `generate_excel_report.py` and find the `create_cost_sheet()` function:

```
python

# Update these values
salt_cost = 50      # Your actual salt cost
fuel_cost = 3.50    # Your actual fuel cost
mpg = 6             # Your truck MPG
wage = 35           # Your driver hourly rate
```

Or make them configurable:

```
python

# At top of file
SALT_COST_PER_TON = 50
FUEL_COST_PER_GALLON = 3.50
# etc.
```

## Traffic Data (Optional Enhancement)

**Current:** Basic traffic API stub

## To Add Real Traffic:

Sign up for a traffic API:

- [Google Maps Traffic API](#)
- [TomTom Traffic API](#)
- [HERE Traffic API](#)

Then update the `fetch_traffic_data()` function (line ~660):

```
python

def fetch_traffic_data(self):
    """Fetch real-time traffic conditions"""
    api_key = "YOUR_API_KEY"

    for road in self.network.roads.values():
        # Query traffic API for this road segment
        url = f"https://api.traffic-provider.com/v1/flow?coords={road.geometry}"
        response = requests.get(url, headers={"Authorization": api_key})

        if response.status_code == 200:
            data = response.json()
            traffic_multiplier = data['speed'] / data['free_flow_speed']
            road.traffic_multiplier = traffic_multiplier
```

This will adjust route times based on real traffic conditions.

---

## TROUBLESHOOTING

### Common Issues & Solutions

#### Issue 1: "ModuleNotFoundError: No module named 'osmnx'"

**Cause:** Package not installed in current environment

**Solution:**

```
bash

conda activate salttrucks
conda install -c conda-forge osmnx -y
```

#### Issue 2: Script runs forever / never completes

**Cause:** Optimization algorithm taking too long

**Solution:** You're using the old version. Make sure you have `PGH_Salt_Trucks_FINAL.py` which uses the optimized algorithm (straight-line distance approximation).

**Check:** Around line 745, you should see:

```
python

# SUPER OPTIMIZED: Use straight-line distance approximation
```

If not, you have the wrong version.

### Issue 3: "FileNotFoundError: optimization\_results.json"

**Cause:** Excel/Map generators can't find the data file

**Solution:** Make sure all 3 scripts are in the same folder, and run them from that folder:

```
bash

cd Desktop
python generate_excel_report.py
```

### Issue 4: Maps show no routes / blank

**Cause:** JSON file has no route geometry data

**Solution:** The main script may have failed during optimization. Check the console output for errors. Rerun the main script.

### Issue 5: "OSM download timeout" or "Connection error"

**Cause:** OpenStreetMap server is slow or overloaded

**Solution:**

```
python

# Add to top of script after imports
import osmnx as ox
ox.settings.timeout = 300 # Increase timeout to 5 minutes
```

### Issue 6: Excel colors/formatting not showing

**Cause:** Old version of openpyxl

**Solution:**

```
bash
```

```
pip install --upgrade openpyxl
```

## Issue 7: Wrong roads being excluded (too many/few)

**Cause:** State road filter is too aggressive or not aggressive enough

**Solution:** Edit the `PENNDOT_MAINTAINED_ROADS` set (lines 39-60) to match your actual state-maintained roads. Remove roads your city IS responsible for, add roads you are NOT responsible for.

---

## ADVANCED CONFIGURATION

### Shift Length & Driver Rules

**Location:** Lines 243-257

```
python
class SaltTruck:
    def __init__(self):
        self.driver_shift_hours = 8.0      # Max shift before overtime
        self.break_hours_interval = 4.0    # Break every 4 hours
        self.break_duration_minutes = 30.0 # 30 min breaks
```

### Customize:

- Union rules: Adjust shift hours
- DOT regulations: Adjust break intervals
- Local policy: Change break duration

### Salt Application Rates

**Location:** Lines 202-220

```
python
```



```
def calculate_salt_rate(self, weather: WeatherConditions):
    # Base rate: 300 lbs per lane mile
    base_rate = 300

    # Adjust for temperature
    if weather.temperature_f < 15:
        rate = base_rate * 1.5 # More salt when very cold
    elif weather.temperature_f < 25:
        rate = base_rate * 1.2
    else:
        rate = base_rate
```

### Customize for your region:

- Colder climates: Increase base rate
- Pre-treatment: Add pre-treatment mode
- Different salt types: Adjust rates for brine vs. rock salt

### Multi-Pass Strategy

**Location:** Lines 1274-1292

```
python

# Calculate if multiple passes needed
passes_needed = 1
if weather.precipitation_type == "heavy_snow":
    passes_needed = 2
elif weather.precipitation_type == "ice_storm":
    passes_needed = 3
```

### Customize:

- Define your storm categories
- Adjust pass requirements
- Add time between passes

### Algorithm Tuning

**Location:** Lines 745-770 (route optimization)

```
python
```

```
# Maximum candidates to check per iteration
```

```
MAX_CANDIDATES = min(50, len(candidate_roads))
```

### Trade-offs:

- **Higher number (100):** Better routes, slower runtime
- **Lower number (25):** Faster runtime, slightly worse routes
- **Sweet spot:** 40-60 for most cities

### Priority Weights

**Location:** Lines 680-690

```
python
```

```
def calculate_priority_score(self, road):  
    # Lower score = higher priority  
    base_score = road.priority * 100  
  
    # Adjust for road type  
    if road.highway_type == "primary":  
        base_score -= 20  
  
    return base_score
```

### Customize:

- Add special categories (school zones, etc.)
- Weight by traffic volume
- Incorporate accident history

---

## MAINTENANCE & UPDATES

### Regular Maintenance Tasks

#### Monthly (During Winter Season)

- ☐ Run test optimization to ensure system works
- ☐ Verify depot locations haven't changed
- ☐ Check for new roads in OpenStreetMap
- ☐ Update cost assumptions (fuel prices, wages)

## Annually (Pre-Season)

- ☐ Review and update priority road list
- ☐ Verify fleet size and depot assignments
- ☐ Test with sample weather scenarios
- ☐ Train new staff on system
- ☐ Update documentation with lessons learned

## After Major Events

- ☐ Compare planned vs. actual routes
- ☐ Identify roads that should change priority
- ☐ Adjust time estimates based on actual performance
- ☐ Document any issues encountered

## Updating OpenStreetMap Data

**The system automatically downloads fresh data each time**, but you can improve OSM for your city:

1. Create account at [OpenStreetMap.org](https://OpenStreetMap.org)
2. Add missing roads in your city
3. Tag roads correctly:
  - `highway=primary` for main roads
  - `highway=secondary` for secondary roads
  - `highway=residential` for neighborhoods
4. Add important features:
  - `amenity=hospital`
  - `amenity=school`
  - `amenity=fire_station`

**Why this helps:** The system uses these tags to auto-prioritize roads near critical infrastructure.

## Version Control Best Practices

1. **Save original scripts** in a safe location
2. **Create dated backups** before making changes:

PGH\_Salt\_Trucks\_FINAL\_2026-01-28.py  
PGH\_Salt\_Trucks\_FINAL\_2026-02-15.py

3. **Document changes** in comments:

```
python

# 2026-01-28: Updated depot locations (JJ)
# 2026-02-15: Changed fleet size to 72 trucks (SM)
```

4. **Test thoroughly** after any changes

**Getting Help**

**Python/Technical Issues:**

- [OSMnx Documentation](#)
- [Folium Documentation](#)
- [Stack Overflow](#) (tag: python, osmnx)

**Optimization Questions:**

- [Operations Research Stack Exchange](#)
- Vehicle Routing Problem (VRP) literature
- Local university Operations Research department

**City-Specific Questions:**

- Contact your Public Works department
- Check state DOT maintenance maps
- Review city GIS department data

---

**APPENDIX: FILE REFERENCE**

**Main System Files**

| File                     | Purpose                     | When to Edit            |
|--------------------------|-----------------------------|-------------------------|
| PGH_Salt_Trucks_FINAL.py | Core optimization engine    | Customize for your city |
| generate_excel_report.py | Excel report generator      | Change cost assumptions |
| generate_maps.py         | Map visualization generator | Rarely need to edit     |

| File                      | Purpose                    | When to Edit        |
|---------------------------|----------------------------|---------------------|
| optimization_results.json | Data file (auto-generated) | Never edit manually |

Output Files

| File                      | Contains              | Audience            |
|---------------------------|-----------------------|---------------------|
| salt_truck_report.xlsx    | 6-sheet workbook      | Internal operations |
| map_full_fleet.html       | All truck routes      | Operations/dispatch |
| map_priority_roads.html   | Critical roads only   | Management          |
| map_public_simple.html    | Simplified coverage   | Public/media        |
| map_depot_*.html          | Individual depot maps | Facility managers   |
| map_coverage_heatmap.html | Density visualization | Planning/analysis   |

Configuration Sections (in main script)

| Lines     | Section              | What to Customize                |
|-----------|----------------------|----------------------------------|
| 20-21     | City coordinates     | Your city center and bounds      |
| 32-60     | Jurisdiction filters | State roads to exclude           |
| 202-220   | Salt rates           | Application rates by temperature |
| 243-257   | Truck parameters     | Shift rules, capacity            |
| 680-690   | Priority scoring     | How roads are prioritized        |
| 1102-1110 | Depot configuration  | Locations and fleet size         |

QUICK START CHECKLIST

For someone new using this system:

## **First Time Setup (30 minutes)**

- ☐ Install Anaconda
- ☐ Create `salttrucks` environment
- ☐ Install packages (osmnx, folium, openpyxl)
- ☐ Download the 3 Python scripts
- ☐ Verify installation with test command

## **Before Each Storm Event (15 minutes)**

- ☐ Open Anaconda Prompt
- ☐ Activate environment: `conda activate salttrucks`
- ☐ Run main script in Spyder (5 min runtime)
- ☐ Generate Excel report (30 sec)
- ☐ Generate maps (1 min)
- ☐ Review outputs for accuracy

## **Customization for Your City (1-2 hours first time)**

- ☐ Update city coordinates
- ☐ Find and add depot locations
- ☐ Identify state-maintained roads to exclude
- ☐ Update cost assumptions
- ☐ Test run with sample data
- ☐ Review outputs and adjust as needed

## **Ongoing Use (5-10 minutes per event)**

- ☐ Run system before each storm
- ☐ Export Excel to share with management
- ☐ Post public map on website/social media
- ☐ Email depot maps to facility managers
- ☐ Track actual vs. predicted performance

---

## **CONTACT & SUPPORT**

### **Documentation Updates**

This guide was created: January 28, 2026

For the latest version, contact your GIS or IT department.

## Training Resources

Recommended training for new users:

1. **Basic Python** (Codecademy, free course, 10 hours)
2. **Excel Basics** (LinkedIn Learning, 2 hours)
3. **GIS Fundamentals** (Coursera, optional, 4 weeks)

## System Limitations

Be aware:

- **Real-time tracking:** This system plans routes, it doesn't track trucks in real-time
  - **Traffic data:** Basic traffic integration; upgrade for live traffic
  - **Weather updates:** Manual rerun needed if weather changes significantly
  - **Map accuracy:** Depends on OpenStreetMap data quality
  - **Optimization:** Uses heuristic algorithm (good but not mathematically optimal)
- 

## CONCLUSION

This system provides a powerful, data-driven approach to winter storm fleet management. With proper setup and maintenance, it will:

- ✓ Save time on route planning
- ✓ Improve coverage equity across neighborhoods
- ✓ Provide accurate cost estimates
- ✓ Enable better communication with residents
- ✓ Support data-driven decision making

The key to success is:

1. **Accurate input data** (depots, fleet size, priorities)
2. **Regular testing** (don't wait for a storm to discover issues)
3. **Continuous improvement** (learn from each event)
4. **Good communication** (share outputs with stakeholders)

**Questions?** Refer to the troubleshooting section or contact your local GIS/IT support team.

---

*System developed for City of Pittsburgh Department of Public Works  
Adaptable for any municipal winter operations program*

**Version 1.0 - January 2026**